

5. ABSTRACT DATA TYPES AND COLLECTIONS

Data types and abstract data types (ADTs)

Requirements, contracts, implementations of ADTs

Design of ADTs in Java

Collections

Abstract data types in the Java class library

Data types

- We classify all data into *data types*.
- Each data type is characterized by:
 - a set of **values**
 - a **data representation** (common to all these values)
 - a set of **operations** (which can be applied uniformly to these values).
- E.g., **boolean** is a built-in data type in Java:
 - the values are **false** and **true**
 - the data representation is a single byte
 - the operations are “| |”, “& &”, “!”, etc.

00000000 = false
00000001 = true

Introducing new data types

- To introduce a new data type, we must define its values, its data representation, and its operations.
- In Java, we use a *class declaration*:
 - The class's **instance variables** determine the values and data representation.
 - The class's **constructors** and **methods** are the operations.

Example: Date data type

- Could represent a Date in several ways (using 3 integers to represent y, m and d, say, or a single integer to represent the day-in-epoch (where 0 represents 1 Jan 2000))
- Good program practice: Application code should not
 - Admit improper values (an exception should be thrown)
 - Access the data representation (i.e. should use information hiding, keep instance variables private, use getters and setters)
 - Be aware of the data representation (we should be able to change the representation and not affect application code)

Abstract data types

- An **abstract data type (ADT)** is characterized by:
 - a set of **values**
 - a set of **operations**.

but is **not** characterized by its data representation
- The data representation is *private*, so application code cannot access it. (Only the ADT's operations can access it.)
- The data representation can be *changed*, with no effect on application code. (Only the ADT's operations must be recoded.)

ADT specification (1)

- Each ADT should have a **contract** that:
 - specifies the set of values of the ADT
 - specifies each operation of the ADT (its name, parameter type(s), result type, observable behavior).
- The contract does *not* specify the data representation, nor how the operations are implemented.
- The **observable behavior** of an operation is its effect as ‘observed’ by the application code. E.g.:

Observable behavior:	sort an array
Algorithm with that behavior:	quick-sort

ADT specification (2)

- **Separation of concerns:**

- The *ADT programmer* undertakes to provide an implementation of the ADT that respects the contract. This programmer is not concerned with what applications the ADT is used for.
- The *application programmer* undertakes to process values of the ADT using only the operations specified in the contract. This programmer is not concerned with how the ADT is implemented.
- Separation of concerns is essential for designing and implementing large systems.

ADT implementation

- An **implementation** of an ADT entails:
 - deciding a data representation (choosing a *data structure*)
 - implementing each operation (choosing an *algorithm*).
- The data representation must be private.
- The data representation must cover all proper values.
 - If the data representation also admits improper values, the operations must avoid generating them.
- The algorithms must be consistent with the data representation.

Collections (1)

- A **collection** is a group of *elements*. E.g.:
 - stack
 - queue
 - list
 - set
 - map.
- A collection is **empty** if it contains no elements.
- The **size** of a collection is the number of elements it contains. (The size may vary.)
- A collection is **bounded** if its size is capped.

Collections (2)

- A collection is **heterogeneous** if its elements may be of different types.
- A collection is **homogeneous** if its elements are all of the same type.
- In Java:
 - A heterogeneous collection is implemented by an ordinary class, with elements of type `Object`.
 - A homogeneous collection is implemented by a *generic class*, with elements of type `E` (where `E` is the generic class's type parameter) or possibly elements consisting of multiple types (like `Pair<K,V>`), but all elements have the same multiple type.

Linking ADTs and Collections

- We will be designing ADTs for various collections
- Will be building our own versions (although often Java has its own version(s))
 - Allows us to see in detail how different implementations work
 - Pros and cons of each implementation (in terms of complexity of operations)
 - In each case we will:
 - create ADT contract using Java interface
 - look at implementations using different data structures
 - compare the complexity of different operations for different implementations

Collection ADTs in the Java class library

- **Class** `java.util.Stack<E>` supports stacks.
- **Interface** `java.util.Queue<E>` supports queues.
- **Interface** `java.util.List<E>` supports lists.
- **Interface** `java.util.Set<E>` supports sets.
- **Packages** `java.awt`, `java.io`, `java.util`, etc., support many other ADTs.

We'll come back to these to compare them to our own implementations for each ADT we consider

Some examples to look at on Moodle

- `Date` - example implementation of a Date class
- `Pair` – interface for homogeneous pair (a simple collection)
- `Pair1`, `Pair2` – two implementations of Pair ADT
- `Person` – used for testing
- `TestPair` – used to test the Pair implementations